

Independent Image Scaling in X and Y Dimensions

Brandon Marquez Salazar

October 27, 2025

1 Introduction and Objectives

This work extends a classical image scaling algorithm to support independent scaling factors along the horizontal (f_x) and vertical (f_y) dimensions. The original algorithm (Program 07 from class) operates with a single scaling factor f , handling two cases:

- $f > 1$: Upscaling using nearest-neighbor interpolation
- $f < 1$: Downscaling using average pooling

The main objective is to generalize this approach to handle four distinct scaling scenarios based on the relationship of f_x and f_y to 1, enabling asymmetric image transformations.

2 Theoretical Background

2.1 Original Scaling Algorithm

The original algorithm processes images using a uniform scaling factor f , where the same transformation applies to both dimensions simultaneously. This approach is efficient but limited to isotropic scaling operations.

Algorithm 1 Original Image Scaling (Escalar)

```
1: procedure Escalar( $I_{in}, I_{out}, f$ )
2:   if  $f > 1.0$  then                                     ▷ Upscaling: nearest neighbor interpolation
3:      $s \leftarrow f$ 
4:      $I_{out} \leftarrow \text{create}(h \cdot s, w \cdot s)$ 
5:     for  $u \in [0, h)$  do
6:       for  $v \in [0, w)$  do
7:          $p \leftarrow I_{in}[u, v]$ 
8:         for  $i \in [u \cdot s, (u + 1) \cdot s)$  do
9:           for  $j \in [v \cdot s, (v + 1) \cdot s)$  do
10:             $I_{out}[i, j] \leftarrow p$ 
11:          end for
12:        end for
13:      end for
14:    end for
15:  else                                                     ▷ Downscaling: average pooling
16:     $s \leftarrow \lfloor 1/f \rfloor$ 
17:    if  $h \not\equiv 0 \pmod s$  OR  $w \not\equiv 0 \pmod s$  then
18:       $I_{out} \leftarrow I_{in}$                                    ▷ Dimension compatibility error
19:    else
20:       $I_{out} \leftarrow \text{create}(h/s, w/s)$ 
21:      for  $u \in [0, h)$  step  $s$  do
22:        for  $v \in [0, w)$  step  $s$  do
23:           $I_{out}[u/s, v/s] \leftarrow \frac{1}{s^2} \sum_{i=u}^{u+s-1} \sum_{j=v}^{v+s-1} I_{in}[i, j]$ 
24:        end for
25:      end for
26:       $I_{out} \leftarrow \text{AjusteDeRango}(I_{out})$ 
27:    end if
28:  end if
29:   $I_{out} \leftarrow \text{convertTo}(CV\_8UC1)$ 
30: end procedure
```

3 Proposed Method: Asymmetric Scaling

The proposed algorithm extends the original approach by introducing independent scaling factors f_x and f_y , resulting in four distinct operational modes:

1. **Case 1:** $f_x > 1$ and $f_y > 1$ (Upscaling both dimensions)
2. **Case 2:** $f_x < 1$ and $f_y > 1$ (Downscaling X, upscaling Y)
3. **Case 3:** $f_x > 1$ and $f_y < 1$ (Upscaling X, downscaling Y)
4. **Case 4:** $f_x < 1$ and $f_y < 1$ (Downscaling both dimensions)

3.1 Algorithm Design

The core challenge lies in properly mapping coordinates between input and output images while applying the appropriate interpolation method for each dimension combination.

Algorithm 2 Asymmetric Image Resize

```
1: procedure Resize( $f_x, f_y$ )
2:    $I_{in} \leftarrow \text{this} \rightarrow \text{image}$ 
3:    $(h, w) \leftarrow \text{dimensions}(I_{in})$ 
4:    $(s_x, s_y) \leftarrow \begin{cases} (f_x, f_y) & \text{if } f_x > 1, f_y > 1 \\ (\lfloor 1/f_x \rfloor, f_y) & \text{if } f_x < 1, f_y > 1 \\ (f_x, \lfloor 1/f_y \rfloor) & \text{if } f_x > 1, f_y < 1 \\ (\lfloor 1/f_x \rfloor, \lfloor 1/f_y \rfloor) & \text{otherwise} \end{cases}$ 
5:    $(h_{out}, w_{out}) \leftarrow \begin{cases} (h \cdot s_y, w \cdot s_x) & \text{if } f_x > 1, f_y > 1 \\ (h \cdot s_y, w/s_x) & \text{if } f_x < 1, f_y > 1 \\ (h/s_y, w \cdot s_x) & \text{if } f_x > 1, f_y < 1 \\ (h/s_y, w/s_x) & \text{otherwise} \end{cases}$ 
6:   if  $\exists$  downscaling AND  $(w \not\equiv 0 \pmod{s_x} \text{ OR } h \not\equiv 0 \pmod{s_y})$  then
7:     return error: "Dimensions not compatible with scaling factors"
8:   end if
9:    $I_{out}.\text{create}(h_{out}, w_{out})$ 
10:  for  $i = 0$  to  $h_{out} - 1$  do
11:    for  $j = 0$  to  $w_{out} - 1$  do
12:       $(u, v) \leftarrow \text{MapCoordinates}(i, j, s_x, s_y, f_x, f_y)$ 
13:       $I_{out}[i, j] \leftarrow \text{ComputePixel}(I_{in}, u, v, s_x, s_y, f_x, f_y)$ 
14:    end for
15:  end for
16:  if  $f_x < 1$  OR  $f_y < 1$  then
17:     $\text{AjusteDeRango}(I_{out})$ 
18:  end if
19:   $\text{this} \rightarrow \text{image} \leftarrow I_{out}$ 
20: end procedure
```

Algorithm 3 Coordinate Mapping Function

```
1: function MapCoordinates( $i, j, s_x, s_y, f_x, f_y$ )
2:   if  $f_x > 1$  AND  $f_y > 1$  then return  $(i/s_y, j/s_x)$ 
3:   else if  $f_x < 1$  AND  $f_y > 1$  then return  $(i/s_y, j \cdot s_x)$ 
4:   else if  $f_x > 1$  AND  $f_y < 1$  then return  $(i \cdot s_y, j/s_x)$ 
5:   elsereturn  $(i \cdot s_y, j \cdot s_x)$ 
6:   end if
7: end function
```

Algorithm 4 Pixel Computation Strategy

```
1: function ComputePixel( $I_{in}, u, v, s_x, s_y, f_x, f_y$ )
2:   if  $f_x > 1$  AND  $f_y > 1$  then
3:     return  $I_{in}[\lfloor u \rfloor, \lfloor v \rfloor]$  ▷ Nearest neighbor for both
4:   else if  $f_x < 1$  AND  $f_y > 1$  then
5:      $sum \leftarrow \sum_{y=\lfloor u \rfloor}^{\lfloor u \rfloor + s_y - 1} \sum_{x=\lfloor v \rfloor}^{\lfloor v \rfloor + s_x - 1} I_{in}[y, x]$ 
6:     return  $sum / (s_x \cdot s_y)$  ▷ Pooling in X, interpolation in Y
7:   else if  $f_x > 1$  AND  $f_y < 1$  then
8:      $sum \leftarrow \sum_{y=\lfloor u \rfloor}^{\lfloor u \rfloor + s_y - 1} \sum_{x=\lfloor v \rfloor}^{\lfloor v \rfloor + s_x - 1} I_{in}[y, x]$ 
9:     return  $sum / (s_x \cdot s_y)$  ▷ Pooling in Y, interpolation in X
10:  else
11:     $sum \leftarrow \sum_{y=\lfloor u \rfloor}^{\lfloor u \rfloor + s_y - 1} \sum_{x=\lfloor v \rfloor}^{\lfloor v \rfloor + s_x - 1} I_{in}[y, x]$ 
12:    return  $sum / (s_x \cdot s_y)$  ▷ Pooling in both dimensions
13:  end if
14: end function
```

4 Implementation Challenges and Results

The implementation revealed several critical issues that prevented successful execution:

4.1 Identified Problems

1. **Dimension Calculation Errors:** The output image dimensions are incorrectly computed, resulting in zero-sized images as shown in the output for image C02.
2. **Coordinate Mapping:** The mapping between input and output coordinates contains off-by-one errors and incorrect boundary handling.
3. **Memory Access Violations:** The algorithm attempts to access pixels outside the valid range of the input image.
4. **OpenCV Integration:** Issues with image format conversion and display functions.

4.2 Execution Output

The program output demonstrates the critical failure during execution:

```
1 Nombre:                ../../res/testImg/C01.bmp
2 Número de filas:       480
3 Número de columnas:    640
4 Total de píxeles:      307200
5 Número de canales:     1
6 Intensidad máxima:     nan
7 Intensidad mínima:     nan
8 Nombre:                ../../res/testImg/C02.bmp
9 Número de filas:       0
10 Número de columnas:   0
11 Total de píxeles:     0
12 Número de canales:    1
13 Intensidad máxima:    nan
14 Intensidad mínima:    nan
15 Nombre:                ../../res/testImg/Family.png
16 Número de filas:      1200
17 Número de columnas:   1600
18 Total de píxeles:     1920000
```

```
19 Número de canales: 4
20 Intensidad máxima: nan
21 Intensidad mínima: nan
22 terminate called after throwing an instance of 'cv::Exception'
23 what(): OpenCV(4.12.0) ../modules/highgui/src/window.cpp:973: error: (-215:
    Assertion failed) size.width>0 && size.height>0 in function 'imshow'
24 Aborted (core dumped)
```

The error clearly indicates that the algorithm produced an image with zero dimensions, causing the OpenCV display function to fail.

5 Conclusion and Future Work

The theoretical extension from single-factor to dual-factor scaling introduces significant complexity in coordinate mapping and pixel computation. While the algorithmic design addresses the four operational modes, the implementation suffers from fundamental issues in dimension calculation and memory access.

5.1 Immediate Corrections Needed

1. **Dimension Verification:** Implement robust dimension calculation with proper integer rounding and boundary checks.
2. **Coordinate Validation:** Add bounds checking in the coordinate mapping functions to prevent out-of-range accesses.
3. **Debugging Infrastructure:** Incorporate intermediate result visualization to identify where the dimension calculation fails.
4. **Unit Testing:** Develop test cases for each of the four scaling scenarios with known input-output pairs.

5.2 Future Directions

Future work will focus on implementing the corrections mentioned above and extending the algorithm to support more sophisticated interpolation methods (bilinear, bicubic) for the up-scaling cases, which would improve image quality in asymmetric scaling scenarios.

The complexity of handling independent scaling factors underscores the importance of careful coordinate system design and thorough validation of dimension calculations in image processing algorithms.